

ROS2 Day1 AI 사용 보고서

학번: 2026406076

이름: 정창규

제출일: 2026-09-09

목차

1. AI 사용 목적
2. Task01에서의 AI 활용
3. Task02에서의 AI 활용
4. Task03에서의 AI 활용
5. AI 사용 과정에서 느낀 점

1. AI 사용 목적

이번 ROS2 Day1 과제를 진행하면서 AI는 과제의 정답을 바로 얻기 위한 용도보다는 수업 자료에서 이해되지 않는 부분을 확인하고, 실습 과정에서 발생한 문제의 원인을 찾기 위한 용도로 사용하였다. ROS2를 처음 다루면서 node, topic, publisher, subscriber, Twist 메시지와 같은 개념이 익숙하지 않았기 때문에 수업 자료를 먼저 확인한 뒤 이해되지 않는 내용을 AI에게 질문하였다.

또한 터미널 명령어를 입력하는 과정에서 원하는 결과가 나오지 않거나, 작성한 C++ 및 Python 코드가 어떤 구조로 동작하는지 확인할 때 AI를 활용하였다. AI가 제시한 내용을 바로 과제 결과로 사용하는 것보다 실제로 터미널에서 실행해보고 결과가 맞는지 확인한 뒤 필요한 부분을 다시 질문하거나 수정하는 방식으로 진행하였다.

2. Task01에서의 AI 활용

Task01을 진행하면서 블로그에서 turtlesim 제어에 사용되는 예제 코드를 찾아 참고하였다. 하지만 /turtle1/cmd_vel 토픽과 geometry_msgs/msg/Twist 메시지의 linear.x, angular.z 값이 각각 어떤 역할을 하는지 정확히 이해하지 못해 AI에게 질문하였다.

AI의 설명을 통해 linear.x는 직선 이동 속도, angular.z는 회전 속도를 의미한다는 것을 이해하였다. 이후 직접 값을 변경하면서 거북이의 움직임이 어떻게 달라지는지 확인하였다.

사각형과 삼각형은 직선 이동과 회전을 반복하여 구현하였고, 원은 linear.x와 angular.z를 동시에 적용하여 만들었다. 블로그의 예제 코드를 그대로 사용하는 것보다 각 값이 어떤 역할을 하는지 이해한 뒤 직접 실행해보는 과정이 도움이 되었다.

블로그에서 turtlesim을 움직이는 예제 코드를 찾았는데, /turtle1/cmd_vel 토픽의 linear.x와 angular.z가 각각 어떤 역할을 하는지 잘 모르겠어. 쉽게 설명해줘

쉽게 보면 /turtle1/cmd_vel 토픽은 거북이에게 "어떻게 움직일지" 속도 명령을 보내는 역할을 해. 여기서 geometry_msgs/msg/Twist 메시지의 linear.x와 angular.z 값을 주로 사용해.

linear.x는 거북이의 직선 이동 속도를 의미해. 양수 값을 넣으면 앞으로 이동하고, 음수 값을 넣으면 뒤로 이동해. 0이면 앞뒤로 움직이지 않아. 값이 커질수록 더 빠르게 이동한다고 보면 돼.

angular.z는 거북이의 회전 속도를 의미해. 양수 값을 주면 왼쪽으로 회전하고, 음수 값을 주면 오른쪽으로 회전해. 이것도 값의 크기가 클수록 더 빠르게 회전하고, 0이면 회전하지 않아.

그래서 linear.x에만 값을 주면 거북이가 직선으로 움직이고, angular.z에만 값을 주면 제자리에서 회전하게 돼. 두 값을 동시에 주면 앞으로 이동하면서 동시에 방향도 바뀌기 때문에 곡선이나 원 형태로 움직일 수 있어.

예를 들어 사각형이나 삼각형을 그릴 때는 linear.x로 직선 이동을 한 뒤 angular.z로 방향을 바꾸는 동작을 반복하면 되고, 원을 그릴 때는 linear.x와 angular.z를 동시에 계속 주는 방식으로 만들 수 있어.

정리하면 linear.x는 "앞뒤 이동", angular.z는 "좌우 회전"을 담당한다고 이해하면 가장 쉬워.

📄 📌 🔄 🗑️ ...

3. Task02에서의 AI 활용

Task02를 진행하면서 ROS2 Publisher와 Subscriber 예제 코드를 찾아 참고하였다. 하지만 C++과 Python에서 Publisher와 Subscriber를 각각 어떻게 구성해야 하는지와, 서로 다른 언어로 작성된 노드끼리도 같은 토픽을 이용해 통신할 수 있는지 정확히 이해하지 못해 AI에게 질문하였다.

AI의 설명을 통해 Publisher는 토픽에 데이터를 보내는 역할을 하고, Subscriber는 해당 토픽을 구독하여 데이터를 받아 callback 함수에서 처리한다는 것을 이해하였다. 또한 C++과 Python으로 작성된 노드라도 같은 토픽 이름과 같은 메시지 타입을 사용하면 서로 통신할 수 있다는 점을 알 수 있었다.

이후 Int32, String, Float32, Bool 메시지를 이용하여 C++과 Python 각각에 Publisher와 Subscriber를 작성하고 직접 실행해보았다. 같은 패키지 내부뿐만 아니라 C++ Publisher와 Python Subscriber, Python Publisher와 C++ Subscriber 사이에서도 정상적으로 데이터가 전달되는 것을 확인하였다.

Task02에서는 예제 코드를 그대로 사용하는 것보다 Publisher와 Subscriber가 어떤 방식으로 연결되고, 서로 다른 언어 사이에서 통신이 가능한 이유를 이해하는 데 AI의 설명이 도움이 되었다.

ROS2 Publisher와 Subscriber 예제 코드를 보고 있는데 C++로 만든 Publisher와 Python으로 만든 Subscriber도 서로 통신할 수 있는지 잘 모르겠고 가능하다면 어떤 조건이 필요한지 쉽게 설명해줘

가능해. ROS2에서는 C++로 만든 Publisher와 Python으로 만든 Subscriber도 서로 통신할 수 있어.

중요한 건 어떤 언어로 작성했는지가 아니라 **같은 토픽과 같은 메시지 형식을 사용하는지야**.

첫 번째로, Publisher와 Subscriber의 **토픽 이름이 같아야 해**.

예를 들어 C++ Publisher가 `/task2/string_data` 라는 토픽으로 데이터를 보내면 Python Subscriber도 같은 `/task2/string_data` 토픽을 구독해야 해.

두 번째로, **메시지 타입이 같아야 해**.

C++ Publisher가 `std_msgs/msg/String` 타입으로 데이터를 보내면 Python Subscriber도 `std_msgs/msg/String` 타입으로 받아야 해. 한쪽은 String이고 다른 쪽은 Int32처럼 타입이 다르면 정상적으로 통신할 수 없어.

세 번째로, 서로 같은 ROS2 환경에서 통신할 수 있어야 해. 같은 컴퓨터에서 실행하거나 같은 네트워크에서 ROS2 통신이 가능하도록 설정되어 있어야 하고, `ROS_DOMAIN_ID` 같은 설정도 서로 맞아야 해.

즉, C++인지 Python인지는 크게 중요하지 않고 **토픽 이름, 메시지 타입, 통신 환경이 같으면 서로 데이터를 주고받을 수 있다고** 이해하면 돼.

그래서 C++ Publisher가 데이터를 보내고 Python Subscriber가 받는 것도 가능하고, 반대로 Python Publisher가 보내고 C++ Subscriber가 받는 것도 가능해.

📄 📌 ↺ 🔄 ...

4. Task03에서의 AI 활용

Task03를 진행하면서 turtlesim을 키보드로 제어하는 예제 코드를 찾아 참고하였다. 하지만 WASD 키 입력을 받아 거북이를 움직이면서 삼각형, 사각형, 원을 일정한 모양으로 그리는 방법이 잘 이해되지 않아 AI에게 질문하였다.

AI의 설명을 통해 키를 누를 때마다 일정한 시간 동안 직진하거나 회전하도록 동작을 정하면 도형을 더 일정하게 그릴 수 있다는 것을 이해하였다. 이후 직선 이동과 회전을 하나의 함수로 처리하고, 도형마다 필요한 회전 시간과 이동 시간을 다르게 설정하였다.

또한 turtlesim의 set_pen 서비스를 이용하면 선의 색상과 굵기도 변경할 수 있다는 점을 확인하였다. 실제 코드에서는 삼각형, 사각형, 원마다 서로 다른 색상과 굵기를 적용하여 구분되도록 하였다.

Task03에서는 AI를 통해 키보드 입력과 이동 동작을 어떻게 연결해야 하는지 이해할 수 있었고, 직접 실행하면서 회전 시간과 선 굵기 값을 조정해 원하는 결과에 맞게 수정하였다.

ROS2에서 turtlesim을 WASD 키로 움직이는 예제 코드를 보고 있는데 키를 눌러 삼각형 사각형 원을 일정하게 그리려면 직선 이동과 회전을 어떤 방식으로 구현하는 게 좋을까



일정한 모양으로 그리려면 키를 누르고 있는 시간에 따라 움직이게 하기보다는, 키를 한 번 눌렀을 때 정해진 시간 동안 이동하거나 회전하도록 만드는 방식이 좋아.

예를 들어 W를 누르면 일정 시간 동안 직선으로 전진하고, A나 D를 누르면 정해진 시간만큼 회전하도록 만들 수 있어. 이렇게 하면 사람이 키를 얼마나 오래 눌렀는지에 따라 모양이 달라지는 문제를 줄일 수 있어.

삼각형은 한 번을 직선으로 이동한 뒤 약 120도씩 회전하는 동작을 반복하면 되고, 사각형은 한 번을 이동한 뒤 약 90도씩 회전하는 동작을 반복하면 돼. 따라서 삼각형 모드와 사각형 모드에서 회전 시간을 서로 다르게 설정하면 같은 A, D 키를 사용하더라도 필요한 각도로 회전시킬 수 있어.

원은 직선 이동과 회전을 따로 하는 방식보다는 선속도와 각속도를 동시에 주는 방법이 좋아. 앞으로 이동하면서 계속 방향을 바꾸게 하면 자연스럽게 원 형태의 궤적을 만들 수 있어.

정리하면 W, S는 직선 이동에 사용하고 A, D는 회전에 사용하되, 도형마다 이동 시간과 회전 시간을 미리 정해두는 방식으로 구현하면 삼각형과 사각형을 비교적 일정하게 그릴 수 있어. 원은 직선 이동과 회전을 동시에 적용하는 방식으로 구현하면 돼.

5. AI 사용 과정에서 느낀 점

이번 ROS2 Day1 과제를 진행하면서 AI는 처음부터 답을 얻기 위한 용도보다는, 블로그나 예제 코드를 보면서 이해되지 않는 부분을 질문하는 데 주로 사용하였다. 특히 turtlesim의 이동 방식, C++과 Python 사이의 Topic 통신, 키보드 입력을 이용한 도형 그리기처럼 처음에는 구조가 잘 이해되지 않았던 부분을 하나씩 질문하면서 개념을 정리할 수 있었다.

AI의 설명을 통해 내용을 이해한 뒤에는 그대로 사용하는 것이 아니라 실제로 코드를 실행해보고 결과를 확인하였다. 실행 결과가 예상과 다를 때에는 값을 다시 조정하거나 어떤 부분이 문제인지 확인하면서 과제를 진행하였다. 이 과정에서 단순히 코드를 복사해서 사용하는 것보다 각 코드와 값이 어떤 역할을 하는지 이해하는 것이 중요하다고 느꼈다.

또한 AI의 답변이 항상 과제 상황에 정확히 맞는 것은 아니기 때문에 수업 자료와 예제 코드, 실제 실행 결과를 같이 확인하는 과정이 필요하다는 점도 알게 되었다. 앞으로도 AI를 사용할 때에는 결과만 바로 사용하는 방식보다는 먼저 자료를 확인하고, 이해되지 않는 부분을 질문한 뒤 직접 실행하고 확인하는 방식으로 활용할 생각이다.